```
   <s:DropDownList
id="layoutSelector" width="100%"
>
   </s:DropDownList>
```

STEP 2: Create an array of layouts to display in the DropDownList

To display the layouts in a list, we need to have a name for the layout to display in the list, and an instance of that layout object. For this we will create an array of Object instances, with each having two properties, a name to display in the list, and a layout that has a layout object instance.

Here is a sample object:

```
   <fx:Object name="Tiles">
   <fx:layout>
   <s:TileLayout />
   </fx:layout>
   </fx:Object>
```

The Object class is the base of everything in Flex; it is a dynamic class, so you can attach properties – such as 'name' and 'layout' – to it at runtime. We will create an array with multiple of these objects and add it as the data source for our list.

Let us add VerticalLayout and HorizontalLayout. Note that you can configure the objects in these instances as you like. Here is the finished code of an ArrayCollection with these three layouts:

```
   <s:ArrayCollection id="layouts">
   <fx:Object name="Tiles">
   <fx:layout>
   <s:TileLayout />
   </fx:layout>
   </fx:Object>
   <fx:Object name="Vertical">
   <fx:layout>
   <s:VerticalLayout gap="5" />
   </fx:layout>
   </fx:Object>
   <fx:Object
name="Horizontal">
   <fx:layout>
<s:HorizontalLayout gap="4"
/>
   </fx:layout>
   </fx:Object>
   </s:ArrayCollection>
```

Note that this needs to be added inside the `<fx:Declarations>` tag in your applications code as it is not a visual component. We have given it an id of 'layouts' so that it can be used as a source for the DropDownList.

STEP 3: Add the array of layouts as data source for the DropDownList

To add this as a data source for our list, we just need to bind it to the dataProvider property. We could have also chosen to place this entire list inside the `<s:DropDownList>` tag, which would have had the same effect.

To have it pick up the names from the objects and display them in the list, we use the labelField property. This property can be used to tell the List component which property of the list items is to be displayed in as a label.

We will also initialize it with a selectedIndex of "0" so that the first layout comes selected by default. Here is the final code:

```
   <s:DropDownList
id="layoutSelector"
labelField="name" width="100%"
selectedIndex="0"
dataProvider="{layouts}">
```

If you run the application now, it will be display the drop-down list with the listed names, however changing it will have no effect. That we save for the final step.

STEP 4: Change the layout with the selection

This step is quite easy. First we remove the layout we have set to prepare to have it set dynamically.

Next all we need to do is bind the layout of the List to the 'layout' property of the currently selected object in the DropDownList. So in the end we need to add the following inside the <s:List> tag::

```
   layout="{layoutSelector.
```

The DropDownList now lists all the layouts

Left: Vertical layout, and right: the horizontal layout

```
selectedItem.layout}"
```

However just to be safe will we have the layout default to the "TileLayout" in case no item is selected. The final code for that is:

```
   <s:List id="imagesList"
dataProvider="{index}"
   width="100%" height="100%"
   layout="{layoutSelector.
selectedItem?layoutSelector.
selectedItem.layout: new Tile-
Layout()}" >
   <s:itemRenderer>
   <fx:Component>
   <mx:Image source="{data.
itemThumbnailImage}" />
   </fx:Component>
   </s:itemRenderer>
   </s:List>
```

You may notice that the layout attribute now reads:

```
   layout="{layoutSelector.
selectedItem?layoutSelector.
selectedItem.layout: new Tile-
Layout()}"
```

Please check to ensure that we always have a valid layout. So if no item is selected (layoutSelector.selectedItem does not have a value), the layout for the List will be a "new TileLayout()" object, while in other cases it will be the 'layout' property of "layoutSelector.selectedItem". Run the application and you will see that the application will change layouts as you change the selection.

Check out the extended tutorial online at http://www.thinkdigit.com/d/fb4_tut1_3 which will show you how to use some more exotic layouts such as a wheel / carousel layout, a flowing layout, coverflow etc.